Capstone Project - Financial News Sentiment Analysis and Price Modelling

James Cook University Cairns

Hunter Kruger-Ilingworth (14198489)

April 13, 2025

Contents

List of Figures

List of Tables

Abstract

This report investigates the feasibility of Australian stock exchange (ASX) price prediction using news article sentiment. Presented is an overview of the issue, a comparison oif various textual sources, a literature review of the subject, and the detailing of development of a machine learning model to predict stock price movements, integrating sentiments and features extracted from smallcaps articles. This report presents a pipeline that collects news articles using web scraping techniques and processes them through NLP systems to extract meaningful insights. All code and reporting files are available on GitHub.

Introduction

Predicting the movements of the stock market is a long standing challenge. Its erratic and volatile nature leads many to believe that predicting it is a fools game, with even experts in the field unable to consistently outperform the market [6–8].

Many efforts have been made to predict stock price movements, with a variety of methods employed, with machine learning being particularly popular in recent years [1]. A literature review conducted by [1] analysed 81 papers from January 2015 to June 2020. This literature review found that support vector machines (SVM) were the most popular method of stock price prediction, with 26.13% of the analysed papers using them, followed by random forest (RF) (15.32%), and long short term memory (LSTM) (13.51%) (fig. 1).

The Efficient Market Hypothesis (EMH) claims that asset prices quickly reflect all available information [6]; it is also believed herd behavior and emotions can drive price swings beyond fundamentals [9]. It therefore follows that incorperating information and sentiment from sources across the internet may improve the performance of stock price prediction models to addess this age old problem. Online financial discourse is heavily concentrated across a few key domains, including social platforms (e.g., Reddit's r/WallStreetBets), news aggregators (e.g., Yahoo Finance), and article-based financial outlets (e.g., Bloomberg), and market announcements [10]. These domains offer continuous textual data that offer differing levels of up to date information and consumer sentiment. Figure 2 shows that a vast majority of the studies analysed in the literature review by [1] used only techincal indicators, and not taking advantage of news data - showing a potential gap in the literature.

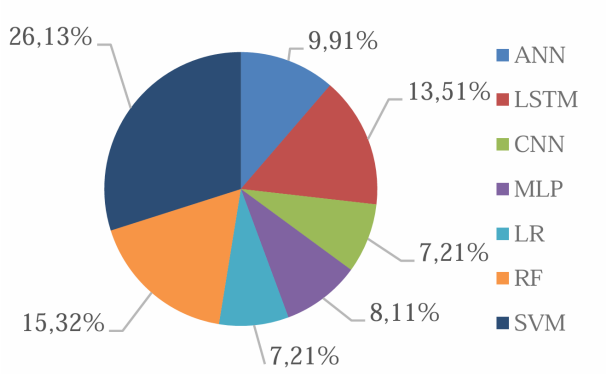

Fig. 1. Proportion of Each Stock Price Prediction method [1]

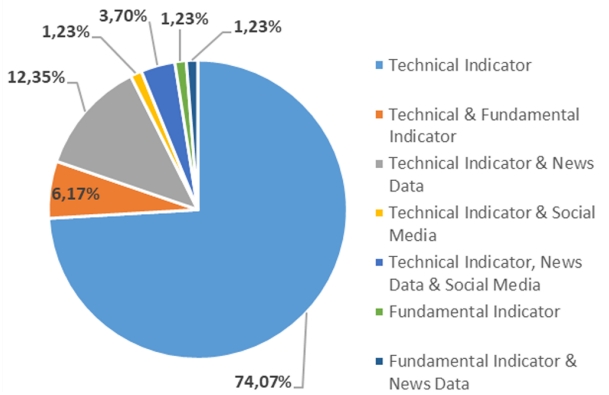

Fig. 2. Proportion of Each of the Studies' Data Source [1]

Web Crawler

Data Type Selection

Several data sources were considered for this investigation. ASX market announcements [10], though frequent and direct, are often irrelevant, provided in PDF format unsuited to text mining, and restricted by ASX's terms of use [11]. Social media platforms like Twitter and Reddit offer high-volume, unstructured content but suffer from slang, spelling errors [12], and questionable reliability [13].

In contrast, financial news articles are typically factual, well-written, and longer in length. Web-based sources often provide a more structured format and accessible metadata, making them better suited to text mining. It is for these reasons that financial news articles were selected as the primary data source for this investigation.

Website Selection

Table 1 shows the comparison of various websites that contain financial and market-related articles, evaluating their suitability for scraping. Copyright is a key legal consideration when using original written works. Under Section 113P of the Copyright Act 1968, Australian copyright law permits educational institutions to copy and communicate works for educational purposes under the 'fair dealing' provisions [14].

Websites were assessed based on their suitability for article scraping, with key criteria including explicit permission in terms of use, presence of disallow/allow directives in `robots.txt`, use of scraping inhibiting in the form of Cloudflare [15], and article structure (e.g. presence of stock codes or site metadata). As shown in Table 1, many domains impose restrictions on automated data collection, and were therefore ruled out. From a sampling design perspective, sites with structured article metadata and consistent tagging (e.g. Trading Economics, Stock Head) were preferred as the metadata tagging could be used for filtering relevant articles and linking content to specific stocks or sectors. Conversely, general news outlets with mixed content and unstructured articles (e.g. ABC News, Yahoo Finance) were excluded due to increased noise and ambiguity in topic association.

A comprehensive summary of the websites considered is shown in table 1. It was concluded that the *Small Caps* website was the most suitable option for scraping. This is due to its static structure, its comprehensive sitemap, and lack of restrictions on scraping. The content is predominantly stock-focused, removing the need for filtering, and articles are consistently tagged with stock codes, providing useful metadata for downstream processing. However, using a single source limits dataset diversity and may bias coverage towards small-cap companies, or a particular sector, or reflect the opinions of a few authors. This drawback could not be easily addressed, as no websites could be found that met the criteria. The possibility of sample bias is investigated and addressed in later sections.

Website Name (URL hyperlinked)	**Scraping Permitted in Terms of Use?**	**Allows Article Scraping in Robots.txt?**	**Uses Cloudflare?**	**General Notes**
Market Index	No	Yes	No	
Yahoo Finance	No	No	No	
Trading Economics	Yes	Yes	No	Highly unstructured. General politics and economics also included — would require extensive filtering
Financial Review	No	No	No	
The Motley Fool	No	Yes	No	
Sydney Morning Herald	No	No	No	
The Market Online	No	Yes	No	
ABC News	Yes	Yes	No	Regular news mixed with financial news, difficult to use, untagged stock market
Small Caps	Yes	Yes	No	Sitemap highly conducive to scraping. Stock codes provided in articles. Static web content also conducive for scraping
Morning Star	No	Yes	No	
Australian Stock Report	Yes	Yes	No	Unstructured in comparison to other article sites
Stock Head	Yes	Yes	No	Sitemap provided that is conducive to scraping, but no list of stock codes provided in each article
The Australian	No	No	No	
Hot Copper	No	Yes	No	

Table 1: Comparison of financial and market-related websites for article scraping suitability. Green rows indicate favourable conditions for scraping.

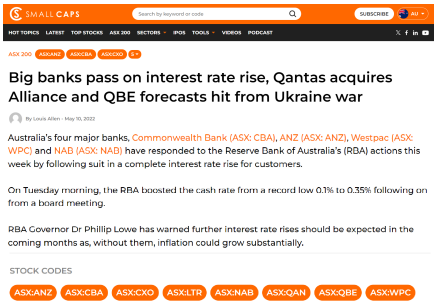

Fig. 3. Example *SmallCaps* Article [2], containing fields such as title, author, date, and list of mentioned stock codes

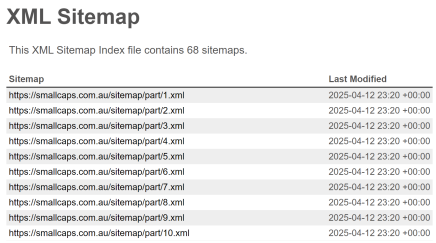

Fig. 4. *SmallCaps'* XML "Sitemap of Sitemaps"

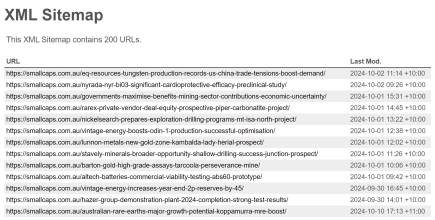

Fig. 5. *SmallCaps'* XML Sitemap Example

Web Scraping

Small Caps was selected in part due to its XML sitemap, which simplifies scraping by directly listing article URLs—removing the need to emulate site navigation using heavier tools like *Selenium*. Instead, the sitemap was parsed using `lxml` with `BeautifulSoup`, which is faster and less resource-intensive for static content. Avoiding *Selenium* also improves resilience to minor site structure changes and avoids issues with popups, though major changes to article page layouts would still require code updates. The `lxml` parser is both fast and capable of parsing XML and HTML, making it well-suited for this approach which involves processing both sitemap files and article content [16].

Although `robots.txt` did not specify a crawl delay, all requests in this scraping process were throttled to one every 3 seconds as an ethical measure to reduce server load. listing 1 shows the sitemap scraping function, which extracts article metadata including title and last-modified date. As shown in listing 2, each paginated sitemap is queried sequentially, filtered for article URLs, and written to a local CSV file. This storage format was chosen for ease of inspection and for easy loading into pandas using `pd.read_csv`.

Table 2 shows the distribution of articles scraped, based on the last modified date in the sitemap.

Listing 1: Definition of xml sitemap scraping function

```python
import requests
from bs4 import BeautifulSoup
from datetime import datetime

def extract_article_metadata_list_from_sitemap(url, sitemap_number):
    """
    Iterate through a sitemap and for each url tag, extracts the URL, title, last modified timestamp, year, month, and day.
    Returns a list of dictionaries, each containing the extracted metadata, for the given sitemap URL.
    """
    response = requests.get(url)
    if response.status_code != 200:
        print(f"Failed to fetch the sitemap. Status code: {response.status_code}")
        return []

    article_observations = []
    soup = BeautifulSoup(response.content, 'lxml-xml')
    for url_tag in soup.find_all('url'):
        loc_tag = url_tag.find('loc')
        lastmod_tag = url_tag.find('lastmod')

        if loc_tag is None:
            continue # skip if <loc> is missing

        loc_tag_text = loc_tag.text.strip()
        last_modified = lastmod_tag.text.strip() if lastmod_tag else None

        # infer report title from url
        title = loc_tag_text.replace("https://smallcaps.com.au", "").replace("-", " ").replace("/", "").capitalize()

        lastmod_timestamp = None
        if last_modified:
            try: # convert last_modified to a datetime object
                lastmod_timestamp = datetime.strptime(last_modified, '%Y-%m-%dT%H:%M:%S%z').timestamp()
            except Exception as e:
                print(f"Error parsing date {last_modified}: {e}")

        article_observations.append({
            "url": loc_tag_text,
            "sitemap_number": sitemap_number,
            "title": title,
            "last_modified_timestamp": lastmod_timestamp,
            "last_modified_year": last_modified[:4] if last_modified else None,  # Year format
            "last_modified_month": last_modified[5:7] if last_modified else None,  # Month format
            "last_modified_day": last_modified[8:10] if last_modified else None  # Day format
        })

    return article_observations
```

Listing 2: Code to use the xml scraping function

```python
import pandas as pd

is_article_url = lambda url: len(url) >= 50  # article titles are included in the url, so we can use this distinction to filter out non-article URLs
sitemap_count = 66
articles = []
for sitemap_number in range(1, sitemap_count + 1):
    print(f"Fetching sitemap {sitemap_number}")
    try:  # there are a list of sitemaps with the names part/1.xml, part/2.xml, we gather all urls from all sitemaps and append to articles list
        sitemap_sub_path = f'sitemap/part/{sitemap_number}.xml'
        article_metadata = extract_article_metadata_list_from_sitemap(f'https://smallcaps.com.au/{sitemap_sub_path}', sitemap_number)
        article_metadata = list(filter(lambda url_data: is_article_url(url_data['url']), article_metadata))
        articles.extend(article_metadata)
    except Exception as e:
        print(f"Failed to fetch sitemap {sitemap_number}. Error: {e}")
df = pd.DataFrame(articles)
df.to_csv('../data/article_metadata.csv', index=False)
```

Year	Number of Observations
2023	10,468
2024	2,374
2025	323
Total	**13,165**

Table 2: Number of Observations per Year As reported by the XML Sitemap Last Modified Date

Scraping the XML sitemaps results in a complete list of article URLs published on *SmallCaps*. Using the `requests` library and `BeautifulSoup` with the `lxml` parser, each article URL was parsed for content. As shown in listing 3, relevant fields were used to extract metadata into a dictionary containing the title, author, date, sector, stock codes, and body text. Table 3 shows an example output from this process - where each of these keys were then converted into columns of a pandas dataframe.

To allow for incremental scraping and avoid reprocessing, existing scraped articles were saved via CSV and excluded in subsequent runs. Requests were throttled with a 3-second delay between each fetch to reduce server load. The scraping code can be seen in listing 4.

Listing 3: Article Scraping function

```python
import re
import urllib.request
from bs4 import BeautifulSoup

def extract_html_from_url(url):
    """
    given a url, return a soup object which contains the html content of the page
    """
    request = urllib.request.Request(url, headers={'User-Agent': 'Mozilla/5.0'})
```

Key	Value
'title'	'Big banks pass on int...'
'author'	'Louis Allen'
'date'	'May 10, 2022'
'sector'	'ASX200'
'stock_codes'	['ASX: CBA', 'ASX: ANZ',...]
'document'	"Australias four maj..."

Table 3: Parsed dictionary example converted into a table

```
    html_content = urllib.request.urlopen(request).read().decode('utf-8')
    soup = BeautifulSoup(html_content, 'lxml')
    return soup

def parse_html_to_dictionary(soup_obj):
    """
    Parse the HTML content of a Small Caps article into a dictionary, extracting the title, author, date, sector, stock codes, and document.
    """

    title_tag = soup_obj.find("h1", class_="c-post-header__title")
    if title_tag: # Parse the title from the <h1> element;
        title = title_tag.get_text(strip=True)
    else: # fall back to <title> if not found.
        title = soup_obj.title.get_text(strip=True) if soup_obj.title else None

    # parse the author from the header
    author_tag = soup_obj.find('a', class_='c-post-header__author-link')
    author = author_tag.get_text(strip=True) if author_tag else None

    # parse the date from the first <time> tag
    date_tag = soup_obj.find('time')
    date_string = date_tag.get_text(strip=True) if date_tag else None

    # parse the sector from the navigation
    sector = None
    nav_section = soup_obj.find("div", class_="c-post__navigation")
    if nav_section:
        sector_tag = nav_section.find("a", class_="c-post__navigation-link")
        sector = sector_tag.get_text(strip=True) if sector_tag else None

    # parse the stock codes
    stock_codes = []
    # look for any div whose class attribute contains "c-tags"
    tags_container = soup_obj.find(
        lambda tag: tag.name == "div" and tag.get("class") and any("c-tags" in c for c in tag.get("class"))
    )
    if tags_container:
        stock_text_span = tags_container.find("span", class_="c-tags__text")
        if stock_text_span and "Stock Codes" in stock_text_span.get_text():
            # Try to find an inner container if available
            inner = tags_container.find("div", class_="c-tags__inner")
            if inner:
                stock_codes = [a.get_text(strip=True) for a in inner.find_all("a", class_="c-tags__item c-tags__item--link")]
            else:
                stock_codes = [a.get_text(strip=True) for a in tags_container.find_all("a", class_="c-tags__item c-tags__item--link")]

    # parse the article body
    body_tag = soup_obj.find("div", class_="c-rich-text c-post__rich-text")
    if body_tag:
        # Use "\n" as separator to preserve paragraph breaks
        document = body_tag.get_text(separator = " ", strip=True)
        document = re.sub(r'[^\w\s]', '', document) if document else None # use regex to remove any character that is not a word character or whitespace
    else:
        document = None

    return {
        'title': title,
        'author': author,
        'date_string': date_string,
        'sector': sector,
        'stock_codes': stock_codes,
        'document': document
    }
```

Listing 4: Code to Parse Websites in xml Sitemap

```
import pandas as pd

is_article_url = lambda url: len(url) >= 50  # article titles are included in the url, so we can use this distinction to filter out non-article URLs
sitemap_count = 66
articles = []
for sitemap_number in range(1, sitemap_count + 1):
    print(f"Fetching sitemap {sitemap_number}")
    try:  # there are a list of sitemaps with the names part/1.xml, part/2.xml, we gather all urls from all sitemaps and append to articles list
        sitemap_sub_path = f'sitemap/part/{sitemap_number}.xml'
        article_metadata = extract_article_metadata_list_from_sitemap(f'https://smallcaps.com.au/{sitemap_sub_path}', sitemap_number)
        article_metadata = list(filter(lambda url_data: is_article_url(url_data['url']), article_metadata))
        articles.extend(article_metadata)
    except Exception as e:
        print(f"Failed to fetch sitemap {sitemap_number}. Error: {e}")
df = pd.DataFrame(articles)
df.to_csv('../data/article_metadata.csv', index=False)
```

Data Augmentation

Listing 8 supplements the broad sector labels from *SmallCaps* with more specific ASX industry data [3]. Each stock code was matched to its ASX-reported industry. To reduce sparsity and simplify analysis, a custom mapping was created to group similar industries together—for example, 'Food, Beverage & Tobacco' and 'Household & Personal Products' were both mapped to 'Consumer Staples' Figure 8 shows the resulting sector breakdown.

Additional Data Collection

For stock price prediction, time series data for each stock mentioned in the articles was required. Due to the complex nested structure and column types, the data was saved as a Parquet file rather than CSV to preserve pandas data types. The `yfinance` library was used, with the documentation used to limit the amount of requests when requesting from the yahoo finance API. The source code can be seen in listing 9

Data Wrangling and Feature Extraction

Exploratory Data Analysis - Sample Distribution

Table 3 shows the structure of the dataset. A total of 1962 articles were collected from Smallcaps. Figure 6 shows the time distribution over time, covering all of 2024, late 2023, and early 2025. This time window may be a limitation, as older articles could exhibit different characteristics not captured in the current dataset. Figure 7 depicts the document length distribution, with a mean word count of 462. The dataset size and document length are sufficient for training machine learning models, particularly when using pretrained transformers such as BERT [17] or FinBERT [18].

Detailed earlier was the addition of the sector as reported by the ASX [3] (Listing 8). These labels, along with those reported by Smallcaps, are shown in fig. 8. This reveals that most articles fall within the mining sector, indicating a sample bias. While ASX labels offer greater context, the larger amount of labels introduces sparsity, with several sectors having very few observations.

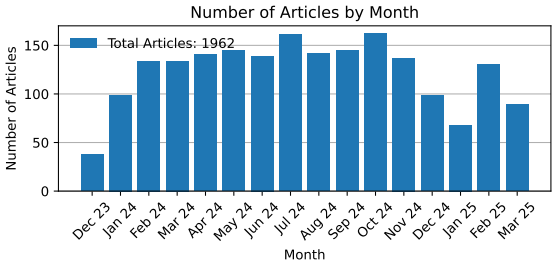

Fig. 6. Quantity of Scraped Articles per Month

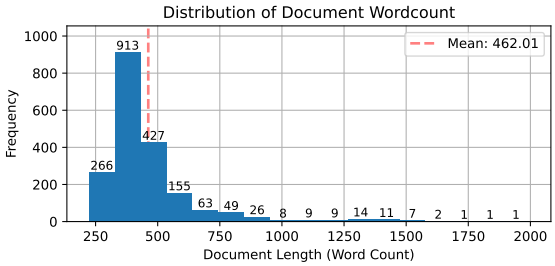

Fig. 7. Distribution of Document Length

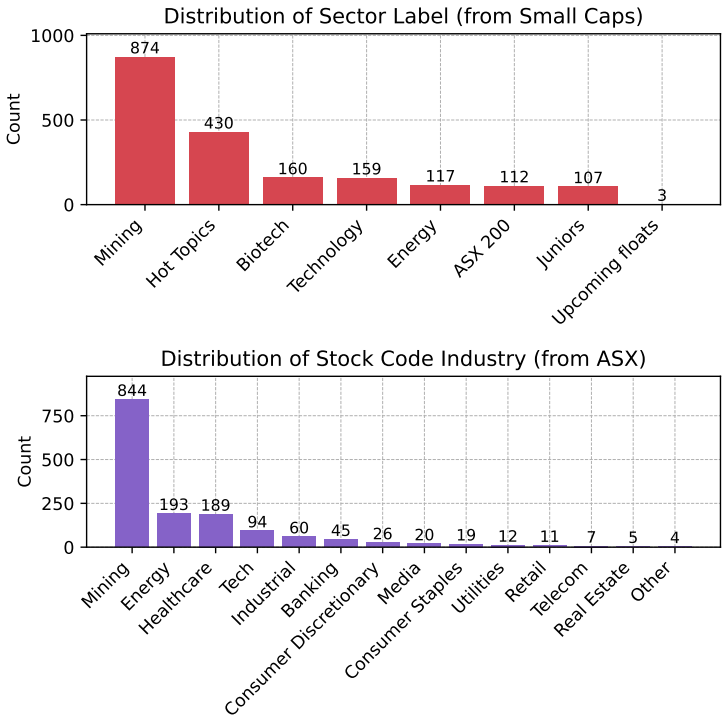

Fig. 8. Distribution of Articles by Sector, using the *smallcaps* industry label and the ASX sector label [3]

Exploratory Data Analysis - Text Corpus

Figure 10 shows the most common words in the articles, whilst removing stop words using `Sklearn CountVectorizer`. It can be seen that past general terms like company, project and asx, mining and resource terms are the most prevailant in the corpus, further demonstrating the mining sampling bias highlighted in fig. 8

Figure 11 shows the PCA biplot of the TF-IDF vectorised article corpus, generated using `SKlearn`'s `PCA` and `TfidfVectorizer` modules. It can be seen that even with two components, the plot shows some level of clustering by sector, suggesting that term usage differs meaningfully between them, suggesting that classification of articles by sector is plausible, but is outside the scope of this investigation.

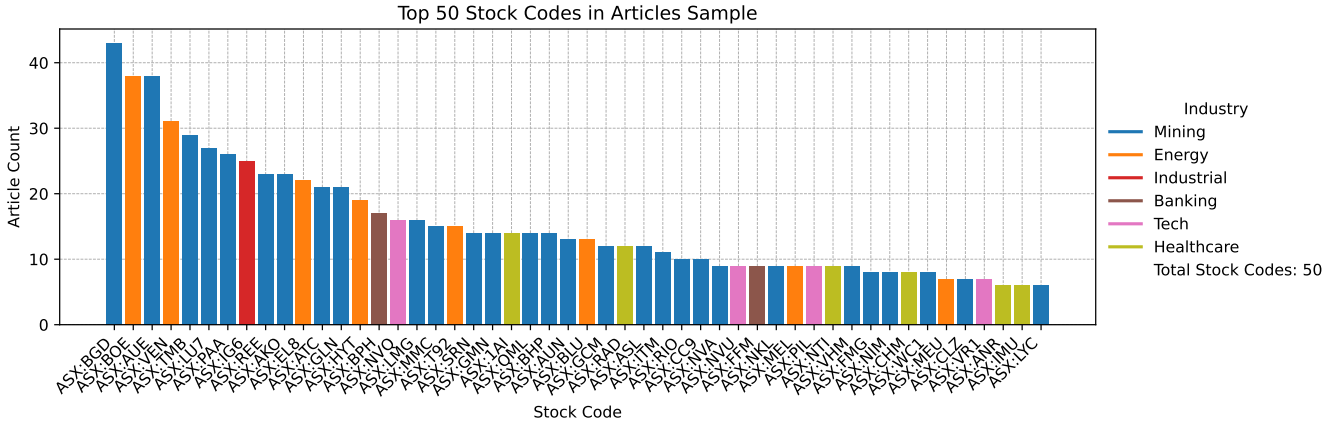

Fig. 9. Top mentioned stock codes in scraped articles

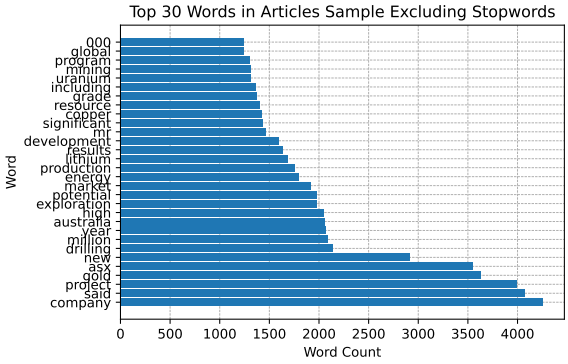

Fig. 10. Frequency of the top words in the scraped articles

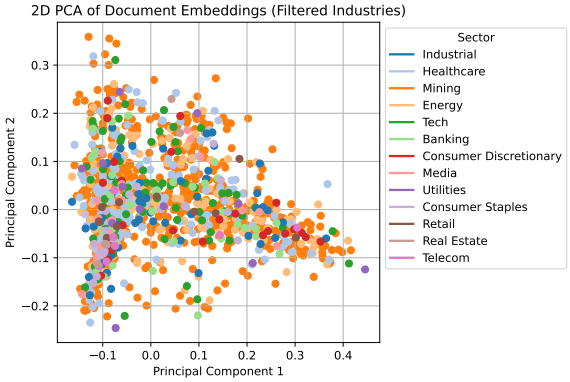

Fig. 11. Principle Component Analysis Biplot of the TF-IDF vectorised article corpus

Feature Extraction

Sentiment Analysis

Article sentiment is modelled using VADER (Valence Aware Dictionary and sEntiment Reasoner) [19], which has shown promising results in similar contexts [13]. The implementation, shown in listing 5, was adapted with reference to [20]. Lemmatisation was considered, as done by Heiden [13], but was ultimately excluded. The `nltk` implementation reduced performance, as VADER relies on common word forms that may differ from lemmatised tokens, reducing its ability to match words to sentiments. The implementation of VADER is shown in Listing 5.

Listing 5: VADER sentiment analysis of article

```
from nltk.stem import PorterStemmer
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer # apply lemminization first

analyzer = SentimentIntensityAnalyzer()
def get_vader_sentiment_scores(document, lemminize_document=False):
    """Clean the text, and calculate VADER sentiment scores"""
    if lemminize_document:
        document = [PorterStemmer().stem(word) for word in document.split()]
    return analyzer.polarity_scores(document)

sentiment_scores = articles_df['document'].apply(get_vader_sentiment_scores)

sentiment_scores_df = sentiment_scores.apply(pd.Series).rename(
    columns={'pos': 'vader_positive', 'neg': 'vader_negative', 'neu': 'vader_neutral', 'compound': 'vader_compound'}
)
articles_df = pd.concat([articles_df, sentiment_scores_df], axis=1)
```

VADER's rule-based approach relies on a fixed lexicon and lacks the capacity to capture context-dependent sentiment. In contrast, BERT leverages deep contextual embeddings and a bidirectional architecture to model nuanced meaning across multi-word expressions. Pretrained on large-scale text such as Wikipedia, BERT can recognise proper nouns and entities, making it effective for disambiguating stock names, tickers, and countries [17]. FinBERT is a domain-specific variant, fine-tuned on financial texts to better capture sentiment in this context [21], and was accessed via the HuggingFace library [18]. However, BERT-based models require labelled data for fine-tuning, making large-scale training infeasible without extensive annotation. To address this, we utilise the `finbert-tone` pretrained transformer, trained on 10,000 manually annotated sentences (positive, negative, neutral) from financial analyst reports [22]. Listing 6 shows the implementation of the `finbert-tone` model.

Listing 6: BERT sentiment analysis of articles

```
from transformers import BertTokenizer, BertForSequenceClassification
from transformers import pipeline

finbert = BertForSequenceClassification.from_pretrained('yiyanghkust/finbert-tone',num_labels=3)
tokenizer = BertTokenizer.from_pretrained('yiyanghkust/finbert-tone')
nlp = pipeline("sentiment-analysis", model=finbert, tokenizer=tokenizer)
def get_bert_sentiment(doc):
  truncated_text = tokenizer.decode(tokenizer.encode(doc, max_length=510, truncation=True))
  result = nlp(truncated_text)[0]  # Call nlp only once
  return pd.Series({'bert_sentiment_label': result['label'], 'bert_sentiment_score': result['score']})
articles_df[['bert_sentiment_label', 'bert_sentiment_score']] = articles_df['document'].apply(get_bert_sentiment)
articles_df
```

Figure 12 shows a strong skew towards positive sentiment in both VADER and BERT outputs, suggesting *SmallCaps* authors may be less inclined to publish news with negative sentiment. Despite architectural differences, Figure 13 shows high correlation between the two models, with few outliers. VADER provides more granularity, outputting separate positive, negative, and neutral scores along with a compound score. In contrast, the `finbert-tone` model produces a single sentiment label and associated score. These different sentiment outputs will be compared and contrasted in future modelling to asses the difference in their predictive power.

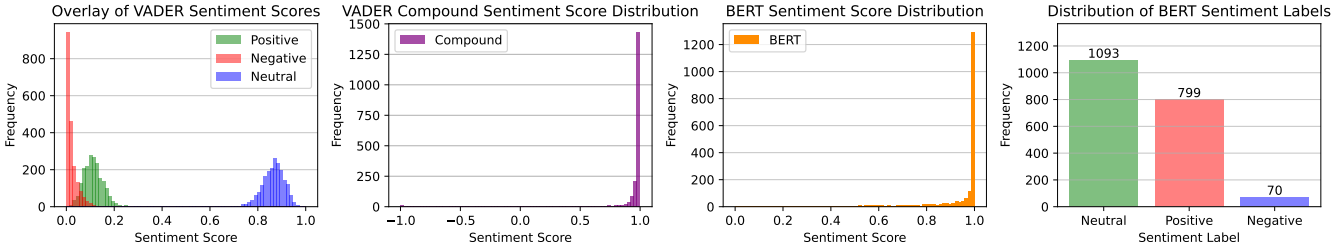

Fig. 12. Distribution of Sentiment Labels for the source data and scores for VADER and BERT

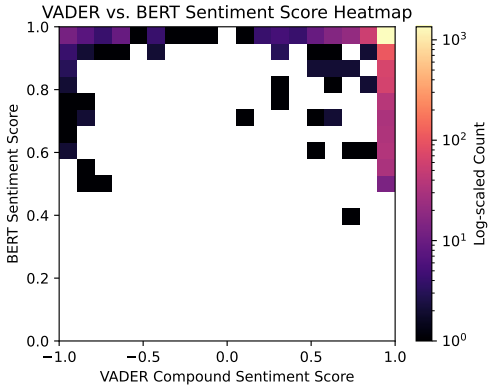

Fig. 13. Heatmap Comparison of sentiment scores of VADER vs BERT

Machine Learning

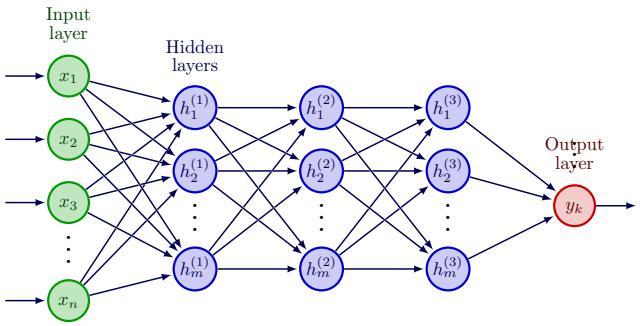

Fig. 14. Typical "FeedForward" Neural Network Topology [4]

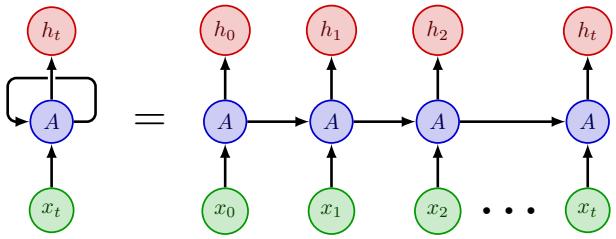

Fig. 15. Recurrent Neural Network (RNN) Topology [5]

A machine learning model was developed to predict stock prices using article data and sentiment features. The literature review conducted concluded that while support vector machines (SVMs) are the most popular in the literature [1], they often underperform in financial contexts due to their sensitivity to parameter tuning [1, 23]. Figures 14 and 15 illustrate the architectures of a typical feedforward neural network and a recurrent neural network (RNN), respectively. RNNs are designed for time series tasks and are thus well suited for stock prediction. RNNs, while powerful, have difficulties with 'memory' due to the number of layers after multiple recursions. Long Short Term Memory networks (LSTMs), a class of RNNs, address this by introducing memory cells with input, output, and forget gates, enabling more effective retention of information across time [24, 25]. Literature in this domain has demonstrated that LSTM models trained on both timeseries price data and sentiment analysis outperform models using price data alone [13, 25].

Listing 7 contains the source code for the single stock LSTM prediction code. Model training was performed in a Google Colab environent [26], due to the increased computational resources made available. Finance data was sourced from Yahoo Finance and merged with pre-collected sentiment data to form a unified dataframe, where each row corresponds to a single trading day, and days with no articles for that particular stock have sentiments set to a default 0 value. Three datasets were constructed: (1) a control set with only historical price data, (2) price data with VADER sentiment scores, and (3) price data with BERT sentiment embeddings. RNNs are trained using sliding windows of past data, so the model learns patterns by looking at every possible sequence of recent days.

Due to class imbalance—particularly the over-representation of mining stocks—no combined model across all stocks was trained. Instead, a separate model was trained for each stock. This approach better captures individual time series characteristics, such as company size, industry dynamics, and news volume. It also inherantly generalises well to other data sources (say, for other markets). While this method increases training time/computational cost, it allows for per-stock tuning and aligns with the approach taken by Heiden [13].

Listing 7: Single stock prediction source code exract (full source code in listing 10)

```
price_data = pd.read_parquet('data/stock_data.parquet')
sentiment_data = pd.read_csv('data/sentiment_data.csv', converters={'stock_codes': ast.literal_eval})
sentiment_data['datetime'] = pd.to_datetime(sentiment_data['date_string'])
sentiment_data

def create_lstm_model(input_shape):
    """Create and compile an LSTM model."""
    optimizer = Adam(learning_rate=0.001)
    model = Sequential()
    model.add(LSTM(64, activation="tanh", return_sequences=True, input_shape=input_shape))
    model.add(LSTM(32, activation="tanh"))
    model.add(Dense(1))
    model.compile(loss='mean_squared_error', optimizer=optimizer)
    return model

```

```python
def build_and_train_model(X_train, Y_train, input_shape, max_epochs, patience):
    """Train an LSTM model with early stopping."""
    model = create_lstm_model(input_shape)
    early_stop = EarlyStopping(monitor='val_loss', patience=patience, restore_best_weights=True)
    history = model.fit(X_train, Y_train, epochs=max_epochs, batch_size=64,
                        validation_split=0.3, callbacks=[early_stop], verbose=1)
    return model, history

def make_predictions(model, X, Y_true):
    """Predict values using a trained model."""
    Y_pred = model.predict(X).reshape(-1)
    return Y_true, Y_pred

def calculate_metrics(y_true, y_pred):
    """Calculate MSE, RMSE, and MAE."""
    mse = mean_squared_error(y_true, y_pred)
    rmse = math.sqrt(mse)
    mae = mean_absolute_error(y_true, y_pred)
    return mse, rmse, mae

def train_and_evaluate_model(stock_name, sentiment_data, max_epochs=20, patience=10,
                             train_proportion=0.7, lookback=30, forecast=1,
                             control_columns=['Close', 'Volume'],
                             sentiment_columns={'Vader': ['vader_negative', 'vader_neutral', 'vader_positive', 'vader_compound'],
                                                'BERT': ['bert_sentiment_score']},
                             output_dir='results'):
    """Train and evaluate models using different dataset variants."""
    data_variants = prepare_data(stock_name, sentiment_data, control_columns, sentiment_columns)

    performance = []
    loss_dict = {}
    pred_dict = {}
    control_Y_true = None
    test_dates = None

    # Process each dataset variant
    for key, dataset in data_variants.items():
        X_train, Y_train, X_test, Y_test, scaler = train_test_split_data(dataset, train_proportion, lookback, forecast)
        input_shape = (X_train.shape[1], X_train.shape[2])
        model, history = build_and_train_model(X_train, Y_train, input_shape, max_epochs, patience)
        loss_dict[key] = history.history['loss']

        Y_true, Y_pred = make_predictions(model, X_test, Y_test)
        # For the Control variant, store test dates and actual values
        if key == 'Control':
            train_size = int(len(dataset) * train_proportion)
            test_data = dataset.iloc[train_size:]
            test_dates = test_data.index[lookback + forecast - 1:][:len(Y_test)]
            control_Y_true = Y_true

        mse, rmse, mae = calculate_metrics(Y_true, Y_pred)
        performance.append({'Model': key + '_' + stock_name, 'MSE': mse, 'RMSE': rmse, 'MAE': mae})
        pred_dict[key] = Y_pred

    plot_training_loss(loss_dict, stock_name, output_dir)
    plot_test_results(test_dates, control_Y_true, pred_dict, stock_name, output_dir)

    return pd.DataFrame(performance)

# example use
performance = train_and_evaluate_model("BHP", sentiment_data, max_epochs=20)
```

A function, `train_test_split()`, was defined to segment each dataset into training and testing partitions. For training, a function `create_lstm_model(input_shape)` was defined to initialise and compile an LSTM model. The model consists of two stacked LSTM layers followed by a dense output layer, and was trained independently on each of the three datasets. The LSTM architecture and training utilised the following hyperparameters:

- Optimiser: Adam ($\alpha = 0.001,\ \beta_1 = 0.9,\ \beta_2 = 0.999,\ \epsilon = 1 \times 10^{-7}$)

These hyperparameters were chosen based on similar implmentations across online sources [27–29]. The Adam optimiser was used due to it being "computationally efficient, having little memory requirement... and is well suited for problems that are large in terms of data/parameters" [29, 30], in addition to being prevalent in existing online implementations [28].

The model architecture consisted of two LSTM layers: the first with 64 units and `tanh` activation, returning sequences; the second with 32 units, also using `tanh`. These layer sizes were based on the model structure presented by Heiden [13]. A final dense output layer with a single neuron used `tanh` activation, rather than the paper's `relu` due to inconsistent convergence during model training. The model was trained using the mean squared error loss function with a batch size of 64, which have been shown to be suitable for LSTMS

[27, 28].

Rather than training for a fixed number of epochs, early stopping was used to halt training if validation loss did not improve for 10 consecutive epochs, reducing the risk of overfitting. The training data was internally split 70/30 for training and validation, with the remaining 30% of the original data reserved for final testing. Models were trained to minimise validation loss, ensuring generalisation to unseen data.

Model performance was compared across the three datasets—the control model, VADER model, and BERT model—to assess the impact of sentiment features on predictive accuracy. Training loss, shown in fig. 16, shows that all models converge, though models using sentiment features took longer, presumably due to the larger input dimentionality.

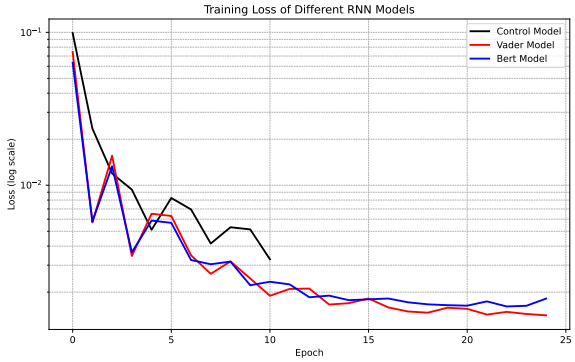

Fig. 16. Training loss vs. epoch plot for the LSTM model example

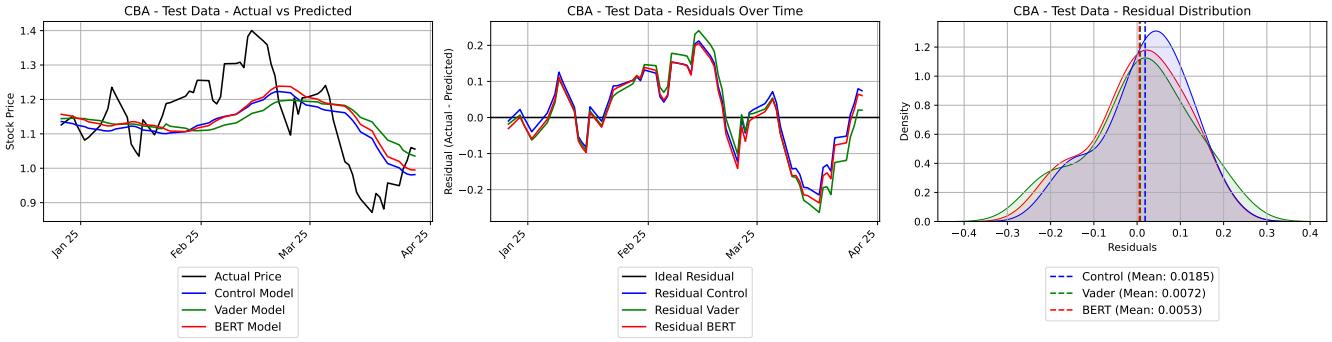

Fig. 17. Model Performance of CBA (Commonwealth Bank) LSTM model

Model	BGD (Mining)	BOE (Energy)	CBA (Banking)	PIL (Tech)	RAD (Healthcare)
Control	0.010008	0.006988	0.010436	0.018697	0.001870
Vader	0.016969	0.006399	0.014232	0.005896	0.001202
BERT	0.017634	0.006984	0.011358	0.015211	0.001345

Table 4: MSE computed across selected stocks and sectors (lower is better).

Figure 17 shows the model performance of the LSTM model trained on the CBA (Commonwealth Bank) data, a typical result seen across multiple stocks. It can be seen that the trend each of these models exhibit are highly correlated, even despite their difference in input features, and that both the time series trend and the overall distribution of their residuals have close resemblance to each other. Table 4 shows a complete summary of the stock codes tested with the LSTM method. It is evident that the incorperating

sentiment data is seen to decrease the performance of the model in the case of BGD, CBA; and improves the performance in the case of BOE, PIL, and RAD, with VADER models having a lower MSE.

This demonstrates that sentiment data does not meaningfully improve the predictive performance of the model. This performance may be largely due to the sparsity of news coverage per stock. As shown in Figure 9, even the most frequently mentioned stock, *ASX:BGD*, appeared in only 40 articles over a 16-month period—equating to just 2.5 articles per month. Such sparsity undermines the reliability and sustainability of sentiment-driven predictive models, a challenge also identified in literature [23]. One solution to the sparsity would be to create synthetic data, which would not be a true representation of prices reflecting real information. As mentioned previously, no domain on the internet meets all the criteria for a suitable data source, so supplementing the article corpus from articles from another website was not a feasible solution, at least within the context of Australian market.

A solution to the data sparsity was posited, to predict the broader ASX200 index and verify if sparsity was the reason for the poor predictive performance of the sentiment. This shift not only mitigates the sparsity issue but also allows the model to capture macroeconomic sentiment and general political news, which are more consistently reported and likely to influence overall market trends (use articles with no stock codes listed).

Performing a similar analysis on the asx200 index (listing 11) and incorperating articles that are in Small-Caps' 'hot topics' category (which is general news that pertains to no industry in particular), and also mention stocks within the ASX200. This way, we can include general politics and other news that may affect the index as a whole. Figure 18 shows the article count for the ASX200 index, showing that the problem of sentiment sparsity is no longer an issue, and days with multiple articles can have their sentiments averaged. Figure 19 shows the training loss vs. epoch plot for the ASX200 model, showing that model convergence occours at the around the 40-50th epoch for all three models. Figure 20 shows the model performance of the ASX200 index models, showing again that the models have similar trends, and that the introduction of sentiments from *SmallCaps* articles does not meaningfully improve the performance of the model, contributing to similar, if not worse performance, as seen with the time series, and overall distribution of the residual (fig. 20) and the numeric results (table 5).

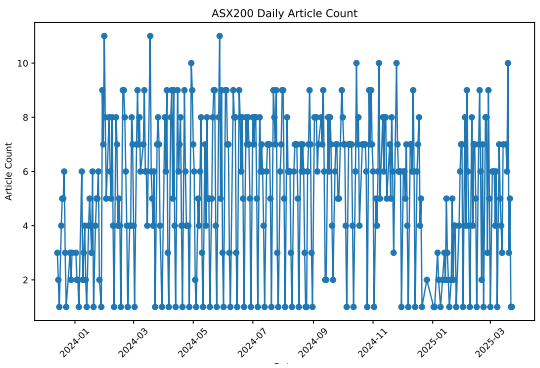

Fig. 18. Article count for the ASX200 index

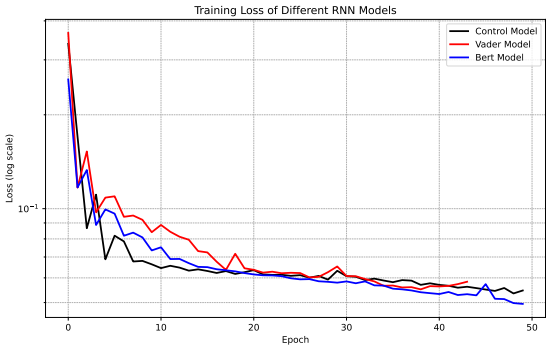

Fig. 19. Training loss vs. epoch plot for ASX200 model

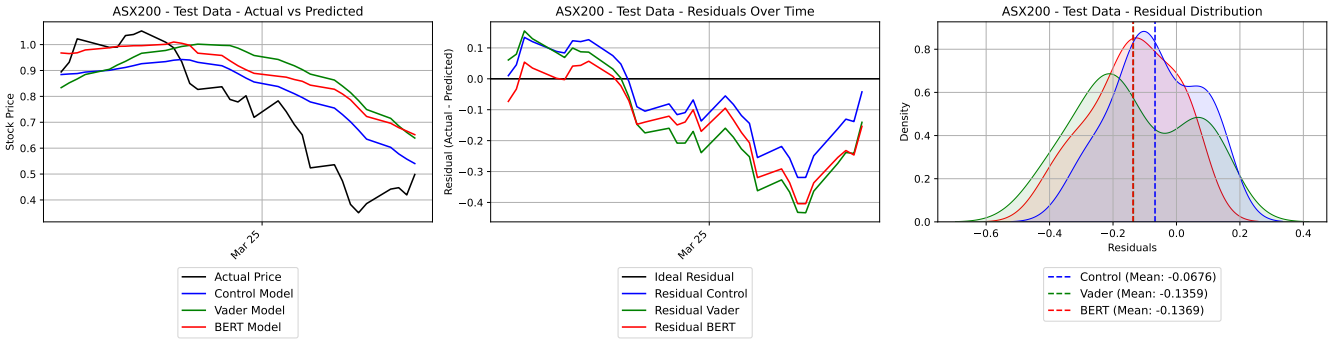

Fig. 20. Model Performance of ASX200 Index Model

Table 5: Model performance metrics for ASX200

Model	MSE	RMSE	MAE
Control	0.022140	0.148794	0.126688
VADER	0.048714	0.220713	0.189641
BERT	0.036432	0.190871	0.151437

Conclusion

This project found that adding sentiment data did not improve the predictive power of the model, than the control model that had no sentiments. One reason for this was the lack of article coverage per stock, which made the sentiment data too sparse to be useful in many cases. Training a separate model for each stock worked well, as it allowed the model to learn each stock's unique patterns, but compounded the spasity issue.

Future work should include other data sources like forums or social media, which can offer faster and more frequent sentiment signals. Prior studies, such as Ko and Chang [25], showed that this can help improve model accuracy. When considering new sources, the same care must be taken to consider copyright and usage terms. Finally, testing other model types like SVMs or random forests could help assess whether simpler models perform better in smaller or noisier datasets.

References

[1] A. Y. Al-Dulaimi, S. N. Al-Saadi, and S. A. Jasim, "The stock exchange prediction using machine learning techniques: A comprehensive and systematic literature review," *Jurnal Ilmu Komputer dan Informasi*, vol. 14, no. 2, pp. 132–150, July 2021, accessed: 2025-04-07. [Online]. Available: https://www.researchgate.net/publication/353002279

[2] L. Allen. (2022, May) Big banks pass on interest rate rise, qantas acquires alliance and qbe forecasts hit from ukraine war. https://smallcaps.com.au/big-banks-interest-rate-rise-qantas-acquires-alliance-qbe-forecasts-hit/. Accessed: 2025-04-02.

[3] ASX Limited, "Company directory," 2024, accessed: 2024-04-08. [Online]. Available: https://www.asx.com.au/markets/trade-our-cash-market/directory

[4] IWN, "Drawing neural network with tikz," https://tex.stackexchange.com/questions/153957/drawing-neural-network-with-tikz, October 2021, answer posted on TeX StackExchange, Accessed: 2025-04-07. [Online]. Available: https://tex.stackexchange.com/questions/153957/drawing-neural-network-with-tikz

[5] user121799, "How do i draw a simple recurrent neural network with goodfellow's style?" https://tex.stackexchange.com/questions/494139/how-do-i-draw-a-simple-recurrent-neural-network-with-goodfellows-style, June 2019, answer posted on TeX StackExchange, Accessed: 2025-04-07. [Online]. Available: https://tex.stackexchange.com/questions/494139/how-do-i-draw-a-simple-recurrent-neural-network-with-goodfellows-style

[6] L. Downey. (2024) Efficient market hypothesis (emh): Definition and critique. Accessed: 2025-04-09. [Online]. Available: https://www.investopedia.com/terms/e/efficientmarkethypothesis.asp

[7] L. Swedroe. (2019) The surprising results from s&p's latest spiva analysis. Accessed: 2025-04-09. [Online]. Available: https://www.advisorperspectives.com/articles/2019/11/18/the-surprising-results-from-s-ps-latest-spiva-analysis

[8] K. Gunnars. (2024) Is it really true that almost no one can beat the market? Accessed: 2025-04-09. [Online]. Available: https://stockanalysis.com/article/can-you-beat-the-market/

[9] T. Greyling and S. Rossouw, "Twitter sentiment and stock market movements: The predictive power of social media," https://cepr.org/voxeu/columns/twitter-sentiment-and-stock-market-movements-predictive-power-social-media, Mar. 2025, voxEU Column, Centre for Economic Policy Research.

[10] ASX Limited, "Today's announcements," https://www.asx.com.au/markets/trade-our-cash-market/todays-announcements, 2025, accessed: 2025-04-02.

[11] ——, "Terms of use," https://www.asx.com.au/legals/terms-of-use, 2023, last updated 12 October 2023. Accessed: 2025-04-02.

[12] S. V. Kolasani and R. Assaf, "Predicting stock movement using sentiment analysis of twitter feed with neural networks," *Journal of Data Analysis and Information Processing*, vol. 8, pp. 309–319, 2020.

[13] A. Heiden and R. S. Parpinelli, "Applying lstm for stock price prediction with sentiment analysis," in *Proceedings of the Brazilian Congress on Computational Intelligence (CBIC)*. Joinville, SC, Brazil: Santa Catarina State University (UDESC), January 2021.

[14] "Copyright act 1968," https://www.legislation.gov.au/Series/C1968A00063, 2024, in force. Administered by Attorney-General's Department.

[15] "Doesitusecloudflare.com," http://www.doesitusecloudflare.com, 2025, accessed: 2025-04-09.

[16] L. Richardson, *Beautiful Soup Documentation*, 2004–2025, retrieved April 27, 2025. [Online]. Available: https://www.crummy.com/software/BeautifulSoup/bs4/doc/#installing-a-parser

[17] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2019, available at https://doi.org/10.48550/arXiv.1810.04805. [Online]. Available: https://arxiv.org/abs/1810.04805

[18] ProsusAI, "Finbert [model]," n.d., retrieved March 27, 2025, from Hugging Face. [Online]. Available: https://huggingface.co/ProsusAI/finbert

[19] C. J. Hutto and E. Gilbert, "VADER: A parsimonious rule-based model for sentiment analysis of social media text," in *Proceedings of the Eighth International Conference on Weblogs and Social Media (ICWSM-14)*. AAAI Press, 2014. [Online]. Available: https://www.researchgate.net/publication/275828927

[20] GeeksforGeeks, "Python sentiment analysis using VADER – using python," December 2024, accessed: 2025-04-08. [Online]. Available: https://www.geeksforgeeks.org/python-sentiment-analysis-using-vader/

[21] D. T. Araci, "Finbert: Financial sentiment analysis with pre-trained language models," Master's thesis, University of Amsterdam, 2019, preprint available on arXiv. [Online]. Available: https://doi.org/10.48550/arXiv.1908.10063

[22] Y. Yang, "Finbert-tone: Fine-tuned financial sentiment model," https://huggingface.co/yiyanghkust/finbert-tone, 2022, accessed: 2025-04-10.

[23] X. Zhang, Y. Li, S. Wang, B. Fang, and P. S. Yu, "Enhancing stock market prediction with extended coupled hidden markov model over multi-sourced data," *arXiv preprint arXiv:1809.00306*, September 2018, 19 pages. Submitted on 2 Sep 2018. Accessed: 2025-04-07. [Online]. Available: https://doi.org/10.48550/arXiv.1809.00306

[24] F. A. Gers, J. Schmidhuber, and F. Cummins, "Learning to forget: Continual prediction with lstm," *Neural Computation*, vol. 12, no. 10, pp. 2451–2471, 2000, also appeared as Technical Report IDSIA-01-99, January 1999.

[25] C.-R. Ko and H.-T. Chang, "Lstm-based sentiment analysis for stock price forecast," *PeerJ Computer Science*, vol. 7, no. 1, p. e408, March 2021.

[26] Google, "Colaboratory," https://colab.google/, 2024.

[27] V. Jain, "Doing multivariate time series forecasting with recurrent neural networks," September 2019, accessed: 2025-04-13. [Online]. Available: https://www.databricks.com/blog/2019/09/10/doing-multivariate-time-series-forecasting-with-recurrent-neural-networks.html

[28] F. M. Aguilera, "Rnns in action: Predicting stock prices with recurrent neural networks," December 2024. [Online]. Available: https://medium.com/thedeephub/rnns-in-action-predicting-stock-prices-with-recurrent-neural-networks-9155a33c4c3b

[29] Keras Team, "Adam class api docs," https://keras.io/api/optimizers/adam/, accessed: 2025-04-07.

[30] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2015, published as a conference paper at the 3rd International Conference for Learning Representations, San Diego, 2015. [Online]. Available: https://doi.org/10.48550/arXiv.1412.6980

[31] R. Aroussi, "yfinance: Yahoo! finance market data downloader," https://yfinance-python.org/, 2024.

Listing 8: Article Data Augmentation Code

```
asx_companies_df = pd.read_csv('../data/ASXListedCompanies.csv')
asx_companies_df['code'] = "ASX:" + asx_companies_df['ASX code']
asx_companies_df

# article_df['stock_codes'] is a cell containing a list of stock code
# create a new column 'stock_code_instustries' which is a list of industries for each stock code found in the asx_comnpaies_df['GICS industry group']
def get_industries(stock_codes):
    if not stock_codes:  # Check if the list is empty
        return None
    industries = asx_companies_df.loc[asx_companies_df['code'].isin(stock_codes), 'GICS industry group']
    return set(industries) if not industries.empty else None

# Create a column for stock code industries (list of industries)
article_df_raw['stock_code_industries'] = article_df_raw['stock_codes'].apply(get_industries)

# Create a column for stock code industry (single industry)
def get_most_popular_industry(industries):
    if industries is None or not industries:  # Handle None or empty set
        return None
    industry_counts = pd.Series(list(industries)).value_counts()
    return industry_counts.idxmax()

article_df_raw['stock_code_industry'] = article_df_raw['stock_code_industries'].apply(get_most_popular_industry)
article_df_raw

# print out a list of all unique stock code industries
unique_industries = set()
for industries in article_df_raw['stock_code_industries']:
    if industries:  # Check if industries is not None
        unique_industries.update(industries)
unique_industries

industry_long_name_to_industry_short_name = {
    "Automobiles & Components": "Auto & Parts",
    "Banks": "Banks",
    "Capital Goods": "Capital Goods",
    "Class Pend": "Classification Pending",
    "Commercial & Professional Services": "Commercial Services",
    "Consumer Discretionary Distribution & Retail": "Retail",
    "Consumer Durables & Apparel": "Durables & Apparel",
    "Consumer Services": "Consumer Services",
    "Energy": "Energy",
    "Equity Real Estate Investment Trusts (REITs)": "Real Estate Invstmt.",
    "Financial Services": "Financial Services",
    "Food, Beverage & Tobacco": "F&B & Tobacco",
    "Health Care Equipment & Services": "Medical Equipment & Services",
    "Household & Personal Products": "Household & Personal Goods",
    "Materials": "Materials",
    "Media & Entertainment": "Media & Entertainment",
    "Not Applic": "Not Applicable",
    "Pharmaceuticals, Biotechnology & Life Sciences": "Pharma & Biotech",
    "Real Estate Management & Development": "Real Estate Mgt & Dev",
    "Semiconductors & Semiconductor Equipment": "Semiconductors",
    "Software & Services": "Software & Services",
    "Technology Hardware & Equipment": "Tech Hardware",
    "Telecommunication Services": "Telecom Services",
    "Transportation": "Transportation",
    "Utilities": "Utilities",
}
broad_sector_map = {
    "Auto & Parts": "Consumer Discretionary",
    "Banks": "Banking",
    "Capital Goods": "Industrial",
    "Classification Pending": "Other",
    "Commercial Services": "Industrial",
    "Retail": "Retail",
    "Durables & Apparel": "Consumer Discretionary",
    "Consumer Services": "Consumer Discretionary",
    "Energy": "Energy",
    "Real Estate Invstmt.": "Real Estate",
    "Financial Services": "Banking",
    "F&B & Tobacco": "Consumer Staples",
    "Medical Equipment & Services": "Healthcare",
    "Household & Personal Goods": "Consumer Staples",
    "Materials": "Mining",
    "Media & Entertainment": "Media",
    "Not Applicable": "Other",
    "Pharma & Biotech": "Healthcare",
    "Real Estate Mgt & Dev": "Real Estate",
    "Semiconductors": "Tech",
    "Software & Services": "Tech",
    "Tech Hardware": "Tech",
    "Telecom Services": "Telecom",
    "Transportation": "Industrial",
    "Utilities": "Utilities",
}

#map the long name to the short name in the dataframe
article_df_raw['stock_code_industry'] = article_df_raw['stock_code_industry'].map(industry_long_name_to_industry_short_name).map(broad_sector_map)
df = article_df_raw
df = df.dropna(subset=['document']).reset_index(drop=True)
df = df.loc[df['sector'] != 'Hot Topics']
df
```

```

# add a datetime column to the dataframe using pd.to_datetime
df['datetime'] = pd.to_datetime(df['date_string'], format='%B %d, %Y', errors='coerce')
df.to_csv('../data/article_scraped_data.csv', index=False)
```

Listing 9: Collection of stock price data from `yfinance` [31]

```
import yfinance as yf
import pandas as pd
import ast
from requests import Session
from requests_cache import CacheMixin, SQLiteCache
from requests_ratelimiter import LimiterMixin, MemoryQueueBucket
from pyrate_limiter import Duration, RequestRate, Limiter


articles_df = pd.read_csv('../data/article_scraped_data.csv', converters={'stock_codes': ast.literal_eval}).dropna(subset=['document'])
articles_df = articles_df[articles_df['stock_codes'].apply(lambda x: len(x) > 0)]
articles_df.dropna(subset=['document'], inplace=True)
stock_codes_unformatted = articles_df['stock_codes'].explode().unique()
def change_stock_string_format(stock_code):
    if stock_code.startswith('ASX:'):
        return stock_code[4:] + '.AX'
    return stock_code
stock_codes = [change_stock_string_format(stock_code) for stock_code in stock_codes_unformatted]

start_date = articles_df['datetime'].min()
end_date = articles_df['datetime'].max()
# --- Download all tickers at once ---

class CachedLimiterSession(LimiterMixin, Session):
    pass
session = CachedLimiterSession(
    limiter=Limiter(RequestRate(2, Duration.SECOND * 5)),  # max 2 requests per 5 seconds
    bucket_class=MemoryQueueBucket,
    backend=SQLiteCache("yfinance.cache"),
)
df = yf.download(stock_codes,
                 start=start_date, end=end_date, period='1d',
                 group_by='ticker',
                 session=session,
                 threads=True,
                 )

df.to_parquet('../data/stock_data.parquet', index=True, engine='pyarrow')
```

Listing 10: Full single stock prediction code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import seaborn as sns
import ast
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import SimpleRNN, Dense, concatenate, Input
import tensorflow as tf
tf.config.run_functions_eagerly(True)
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.metrics import mean_squared_error, mean_absolute_error
import math
import pandas as pd
import os



price_data = pd.read_parquet('data/stock_data.parquet')
sentiment_data = pd.read_csv('data/sentiment_data.csv', converters={'stock_codes': ast.literal_eval})
sentiment_data['datetime'] = pd.to_datetime(sentiment_data['date_string'])
sentiment_data

def get_stock_sentiment(stock_ticker, sentiment_data):
  search_term = f'ASX:{stock_ticker.upper()}'  # Create search term
  filtered_data = sentiment_data[sentiment_data['stock_codes'].apply(lambda codes: search_term in codes)]
  stock_sentiment_df = filtered_data[['datetime', 'vader_negative', 'vader_neutral', 'vader_positive', 'vader_compound', 'bert_sentiment_score']].copy()
  stock_sentiment_df.set_index('datetime', inplace=True)
  return stock_sentiment_df

def get_raw_stock_data(stock_ticker, df=price_data):
  """given the format BHP, returns df['BHP.AX']"""
  return df[f'{stock_ticker.upper()}.AX'].copy()

def prepare_data(stock_name, sentiment_data, control_columns, sentiment_columns):
    """Merge stock price and sentiment data and create dataset variants."""
    sentiment_df = get_stock_sentiment(stock_name, sentiment_data)
    price_df = get_raw_stock_data(stock_name)
    merged_df = pd.merge(price_df, sentiment_df, left_index=True, right_index=True, how='left').fillna(0)

```

```
    data = {}
    data['Control'] = merged_df[control_columns].copy()
    for key, cols in sentiment_columns.items():
        data[key] = merged_df[control_columns + cols].copy()
    return data

def create_dataset(dataset, lookback, forecast):
    """Create lookback dataset for time series prediction."""
    X, Y = [], []
    max_index = len(dataset) - lookback - forecast
    for i in range(max_index):
        X.append(dataset.iloc[i:i + lookback].values)
        Y.append(dataset['Close'].iloc[i + lookback + forecast - 1])
    return np.array(X), np.array(Y)

def train_test_split_data(data_df, train_proportion, lookback, forecast):
    """Split, scale, and create lookback datasets."""
    train_size = int(len(data_df) * train_proportion)
    train_data = data_df.iloc[:train_size].copy()
    test_data = data_df.iloc[train_size:].copy()

    scaler = MinMaxScaler()
    scaler.fit(train_data)
    train_scaled = pd.DataFrame(scaler.transform(train_data), columns=train_data.columns, index=train_data.index)
    test_scaled = pd.DataFrame(scaler.transform(test_data), columns=test_data.columns, index=test_data.index)

    X_train, Y_train = create_dataset(train_scaled, lookback, forecast)
    X_test, Y_test = create_dataset(test_scaled, lookback, forecast)
    return X_train, Y_train, X_test, Y_test, scaler

def create_lstm_model(input_shape):
    """Create and compile an LSTM model."""
    optimizer = Adam(learning_rate=0.001)
    model = Sequential()
    model.add(LSTM(64, activation="tanh", return_sequences=True, input_shape=input_shape))
    model.add(LSTM(32, activation="tanh"))
    model.add(Dense(1))
    model.compile(loss='mean_squared_error', optimizer=optimizer)
    return model

def build_and_train_model(X_train, Y_train, input_shape, max_epochs, patience):
    """Train an LSTM model with early stopping."""
    model = create_lstm_model(input_shape)
    early_stop = EarlyStopping(monitor='val_loss', patience=patience, restore_best_weights=True)
    history = model.fit(X_train, Y_train, epochs=max_epochs, batch_size=64,
                        validation_split=0.3, callbacks=[early_stop], verbose=1)
    return model, history

def make_predictions(model, X, Y_true):
    """Predict values using a trained model."""
    Y_pred = model.predict(X).reshape(-1)
    return Y_true, Y_pred

def calculate_metrics(y_true, y_pred):
    """Calculate MSE, RMSE, and MAE."""
    mse = mean_squared_error(y_true, y_pred)
    rmse = math.sqrt(mse)
    mae = mean_absolute_error(y_true, y_pred)
    return mse, rmse, mae

def plot_training_loss(loss_dict, stock_name, output_dir):
    """Plot the training loss vs epochs for each model."""
    plt.figure(figsize=(10, 6))
    plt.semilogy(loss_dict.get('Control', []), label='Control Model', linewidth=2, color='black')
    plt.semilogy(loss_dict.get('Vader', []), label='Vader Model', linewidth=2, color='red')
    plt.semilogy(loss_dict.get('BERT', []), label='Bert Model', linewidth=2, color='blue')
    plt.grid(True, which="both", linestyle='--', linewidth=0.5)
    plt.gca().yaxis.set_minor_formatter(plt.NullFormatter())
    plt.xlabel('Epoch')
    plt.ylabel('Loss (log scale)')
    plt.title('Training Loss of Different RNN Models')
    plt.legend()
    if not os.path.exists(output_dir):
        os.makedirs(output_dir)
    plt.savefig(os.path.join(output_dir, f'{stock_name}_model_training_loss.pdf'), format='pdf')
    plt.show()

def plot_test_results(test_dates, actual, predictions, stock_name, output_dir):
    """Create a 3-way subplot for test results: actual vs predicted, residuals over time, and residual KDE."""
    # Compute residuals for each model
    residuals = {}
    for key, pred in predictions.items():
        residuals[key] = actual - pred

    # Create a DataFrame for KDE plotting
    residuals_df = pd.DataFrame(residuals)
    melted_res = residuals_df.melt(var_name='Model', value_name='Residuals')
    model_colours = {'Control': 'blue', 'Vader': 'green', 'BERT': 'red'}

    fig, axes = plt.subplots(1, 3, figsize=(18, 5))

    # Plot 1: Actual vs Predicted (Testing)
    ax1 = axes[0]
    ax1.plot(test_dates, actual, label='Actual Price', color='black')
    ax1.plot(test_dates, predictions.get('Control', []), label='Control Model', color='blue')
    ax1.plot(test_dates, predictions.get('Vader', []), label='Vader Model', color='green')
    ax1.plot(test_dates, predictions.get('BERT', []), label='BERT Model', color='red')
```

```
    ax1.set_title(f'{stock_name} - Test Data - Actual vs Predicted')
    ax1.set_ylabel('Stock Price')
    ax1.grid(True)
    ax1.xaxis.set_major_locator(mdates.MonthLocator(interval=1))
    ax1.xaxis.set_major_formatter(mdates.DateFormatter('%b %y'))
    plt.setp(ax1.xaxis.get_majorticklabels(), rotation=45, ha="right")
    ax1.legend(loc='upper center', bbox_to_anchor=(0.5, -0.2), ncol=1, fontsize=11)

    # Plot 2: Residuals over time (Testing)
    ax2 = axes[1]
    ax2.axhline(0, color='black', linestyle='-', label='Ideal Residual')
    ax2.plot(test_dates, residuals.get('Control', []), label='Residual Control', color='blue')
    ax2.plot(test_dates, residuals.get('Vader', []), label='Residual Vader', color='green')
    ax2.plot(test_dates, residuals.get('BERT', []), label='Residual BERT', color='red')
    ax2.set_title(f'{stock_name} - Test Data - Residuals Over Time')
    ax2.set_ylabel('Residual (Actual - Predicted)')
    ax2.grid(True)
    ax2.xaxis.set_major_locator(mdates.MonthLocator(interval=1))
    ax2.xaxis.set_major_formatter(mdates.DateFormatter('%b %y'))
    plt.setp(ax2.xaxis.get_majorticklabels(), rotation=45, ha="right")
    ax2.legend(loc='upper center', bbox_to_anchor=(0.5, -0.2), ncol=1, fontsize=11)

    # Plot 3: KDE of Residuals (Testing)
    ax3 = axes[2]
    sns.kdeplot(
        data=melted_res,
        x='Residuals',
        hue='Model',
        fill=True,
        alpha=0.1,
        palette=model_colours,
        ax=ax3
    )
    for model, color in model_colours.items():
        mean_res = residuals_df[model].mean()
        ax3.axvline(mean_res, color=color, linestyle='--', label=f'{model} (Mean: {mean_res:.4f})')
    ax3.set_title(f'{stock_name} - Test Data - Residual Distribution')
    ax3.set_xlabel('Residuals')
    ax3.set_ylabel('Density')
    ax3.grid(True)
    ax3.legend(loc='upper center', bbox_to_anchor=(0.5, -0.2), ncol=1, fontsize=11)

    plt.tight_layout()
    if not os.path.exists(output_dir):
        os.makedirs(output_dir)
    plt.savefig(os.path.join(output_dir, f'residuals_{stock_name}.pdf'), format='pdf')
    plt.show()

def train_and_evaluate_model(stock_name, sentiment_data, max_epochs=20, patience=10,
                             train_proportion=0.7, lookback=30, forecast=1,
                             control_columns=['Close', 'Volume'],
                             sentiment_columns={'Vader': ['vader_negative', 'vader_neutral', 'vader_positive', 'vader_compound
    '],
                                                'BERT': ['bert_sentiment_score']},
                             output_dir='results'):
    """Train and evaluate models using different dataset variants."""
    data_variants = prepare_data(stock_name, sentiment_data, control_columns, sentiment_columns)

    performance = []
    loss_dict = {}
    pred_dict = {}
    control_Y_true = None
    test_dates = None

    # Process each dataset variant
    for key, dataset in data_variants.items():
        X_train, Y_train, X_test, Y_test, scaler = train_test_split_data(dataset, train_proportion, lookback, forecast)
        input_shape = (X_train.shape[1], X_train.shape[2])
        model, history = build_and_train_model(X_train, Y_train, input_shape, max_epochs, patience)
        loss_dict[key] = history.history['loss']

        Y_true, Y_pred = make_predictions(model, X_test, Y_test)
        # For the Control variant, store test dates and actual values
        if key == 'Control':
            train_size = int(len(dataset) * train_proportion)
            test_data = dataset.iloc[train_size:]
            test_dates = test_data.index[lookback + forecast - 1:][:len(Y_test)]
            control_Y_true = Y_true

        mse, rmse, mae = calculate_metrics(Y_true, Y_pred)
        performance.append({'Model': key + '_' + stock_name, 'MSE': mse, 'RMSE': rmse, 'MAE': mae})
        pred_dict[key] = Y_pred

    plot_training_loss(loss_dict, stock_name, output_dir)
    plot_test_results(test_dates, control_Y_true, pred_dict, stock_name, output_dir)

    return pd.DataFrame(performance)

# example use
performance = train_and_evaluate_model("BHP", sentiment_data, max_epochs=20)
```

Listing 11: ASX200 index prediction code

```
def prepare_asx200_sentiment_data():
    """Load and aggregate the ASX200 sentiment data."""
    asx200_tickers = ['CBA', 'BHP', 'CSL', 'WBC', 'ANZ', 'FMG', 'NAB', 'MQG', 'GMG', 'WOW', 'WES', 'TLS',
```

```
                    'RIO', 'WDS', 'TCL', 'SQ2', 'ALL', 'COL', 'SCG', 'S32', 'NCM', 'SUN', 'QBE', 'BXB',
                    'FPH', 'COH', 'ASX', 'STO', 'RHC', 'AMC', 'ORG', 'IAG', 'SHL', 'DXS', 'BSL', 'APA',
                    'TWE', 'REA', 'AIA', 'DMP', 'CPU', 'TAH', 'VCX', 'MGR', 'QAN', 'GPT', 'EVN', 'SGP',
                    'BLD', 'MPL', 'JHX', 'LLC', 'SOL', 'ALD', 'WOR', 'AZJ', 'CHC', 'ORI', 'HVN', 'XRO',
                    'SVW', 'SEK', 'TPG', 'MFG', 'IEL', 'FBU', 'ALQ', 'ALX', 'NST', 'BEN', 'WTC', 'BOQ',
                    'RMD', 'IGO', 'CWY', 'AGL', 'ANN', 'CAR', 'AWC', 'VEA', 'IPL', 'WHC', 'QUB', 'A2M',
                    'APE', 'SGR', 'LNK', 'DOW', 'BRG', 'AMP', 'ORA', 'CGF', 'ALU', 'RWC', 'ARB', 'CIA',
                    'OZL', 'ILU', 'FLT', 'BKW', 'BPT', 'GOZ', 'CTD', 'CLW', 'VUK', 'CSR', 'NEC', 'DHG',
                    'MIN', 'BWP', 'BAP', 'PDL', 'RRL', 'HLS', 'IFL', 'PME', 'NHF', 'PMV', 'ABP', 'JBH',
                    'SDF', 'MTS', 'CMW', 'SBM', 'TNE', 'SGM', 'DRR', 'SCP', 'CQR', 'CNU', 'NXT', 'JHG',
                    'WEB', 'ABC', 'IRE', 'CCP', 'NUF', 'CIP', 'NWL', 'INA', 'PPT', 'HUB', 'BGA', 'WPR',
                    'ELD', 'IVC', 'AUB', 'CKF', 'IPH', 'SUL', 'CUV', 'ING', 'SLR', 'CGC', 'BKL', 'PLS',
                    'GOR', 'NAN', 'EML', 'GNC', 'NSR', 'LYC', 'GUD', 'NWS', 'KLS', 'AKE', 'ARF', 'AVZ',
                    'BRN', 'CNI', 'CHN', 'CCX', 'CXO', 'CRN', 'DEG', 'EDV', 'EVT', 'HMC', 'HDN', 'IMU',
                    'LKE', 'LIC', 'LTR', 'MP1', 'NHC', 'NIC', 'NVX', 'PDN', 'PRU', 'PNI', 'PBH', 'RMS',
                    'REH', 'SFR', 'TLX', 'TLC', 'TYR', 'UMG', 'UWL', 'ZIP']
    asx200_tickers = [f'ASX:{ticker}' for ticker in asx200_tickers]

    sentiment_data = pd.read_csv('data/sentiment_data.csv', converters={'stock_codes': ast.literal_eval})
    sentiment_data['datetime'] = pd.to_datetime(sentiment_data['date_string'])

    # Mark articles related to ASX200 and general news
    sentiment_data['is_in_asx200'] = sentiment_data['stock_codes'].apply(lambda codes: any(code in asx200_tickers for code in
      codes))
    sentiment_data['is_general_news'] = sentiment_data['stock_codes'].apply(lambda codes: len(codes) == 0)

    sentiment_data['bert_sentiment_label'] = sentiment_data['bert_sentiment_label'].replace({
        'Negative': -2,
        'Neutral': 0,
        'Positive': 2
    })

    sentiment_columns = ['vader_negative', 'vader_neutral', 'vader_positive', 'vader_compound',
                         'bert_sentiment_label', 'bert_sentiment_score']
    daily_sentiment = sentiment_data.groupby('datetime')[sentiment_columns + ['stock_codes']].agg(
        article_count=('stock_codes', 'count'),
        vader_negative=('vader_negative', 'mean'),
        vader_neutral=('vader_neutral', 'mean'),
        vader_positive=('vader_positive', 'mean'),
        vader_compound=('vader_compound', 'mean'),
        bert_sentiment_label=('bert_sentiment_label', 'mean'),
        bert_sentiment_score=('bert_sentiment_score', 'mean')
    ).reset_index()
    daily_sentiment.set_index('datetime', inplace=True)
    return daily_sentiment

def plot_article_count(daily_sentiment):
    """Plot the number of articles over time."""
    plt.figure(figsize=(10, 6))
    plt.plot(daily_sentiment.index, daily_sentiment['article_count'], marker='o', linestyle='-')
    plt.xlabel('Date')
    plt.ylabel('Article Count')
    plt.title('ASX200 Daily Article Count')
    plt.xticks(rotation=45)
    plt.savefig('results/asx200_article_count.pdf', format='pdf')
    plt.show()

def get_asx200_price_data(start_date, end_date):
    """Download ASX200 index price data using yfinance."""
    price_data = yf.download('^AXJO', start=start_date, end=end_date, period='1d', group_by='ticker', threads=True)['^AXJO'].
      copy()
    return price_data

def merge_asx200_data(price_data, daily_sentiment):
    """Merge ASX200 price data with aggregated sentiment data."""
    merged_df = pd.merge(price_data, daily_sentiment, left_index=True, right_index=True, how='left').fillna(0)
    return merged_df

def prepare_data(merged_df, control_columns, sentiment_columns):
    """Create dataset variants for analysis."""
    data = {}
    data['Control'] = merged_df[control_columns].copy()
    for key, cols in sentiment_columns.items():
        data[key] = merged_df[control_columns + cols].copy()
    return data

def create_dataset(dataset, lookback, forecast):
    """Create lookback dataset for time-series prediction."""
    X, Y = [], []
    max_index = len(dataset) - lookback - forecast
    for i in range(max_index):
        X.append(dataset.iloc[i:i+lookback].values)
        Y.append(dataset['Close'].iloc[i + lookback + forecast - 1])
    return np.array(X), np.array(Y)

def train_test_split_data(data_df, train_proportion, lookback, forecast):
    """Split the data into train/test, scale it, and create lookback datasets."""
    train_size = int(len(data_df) * train_proportion)
    train_data = data_df.iloc[:train_size].copy()
    test_data = data_df.iloc[train_size:].copy()

    scaler = MinMaxScaler()
    scaler.fit(train_data)
    train_scaled = pd.DataFrame(scaler.transform(train_data), columns=train_data.columns, index=train_data.index)
    test_scaled = pd.DataFrame(scaler.transform(test_data), columns=test_data.columns, index=test_data.index)

    X_train, Y_train = create_dataset(train_scaled, lookback, forecast)
```

```
    X_test, Y_test = create_dataset(test_scaled, lookback, forecast)

    # Ensure the shape is [samples, timesteps, features]
    X_train = X_train.reshape(X_train.shape[0], X_train.shape[1], X_train.shape[2])
    X_test = X_test.reshape(X_test.shape[0], X_test.shape[1], X_test.shape[2])
    return X_train, Y_train, X_test, Y_test, scaler

def create_lstm_model(input_shape):
    """Create and compile an LSTM model."""
    optimizer = Adam(learning_rate=0.001, beta_1=0.9, beta_2=0.999, epsilon=1e-07)
    model = Sequential()
    model.add(LSTM(64, activation="tanh", return_sequences=True, input_shape=input_shape))
    model.add(LSTM(32, activation="tanh"))
    model.add(Dense(1, activation='linear'))
    model.compile(loss='mean_squared_error', optimizer=optimizer)
    return model

def build_and_train_model(X_train, Y_train, input_shape, max_epochs, patience):
    """Build and train the SimpleRNN model with early stopping."""
    model = create_lstm_model(input_shape)
    early_stop = EarlyStopping(monitor='val_loss', patience=patience, restore_best_weights=True)
    history = model.fit(X_train, Y_train, epochs=max_epochs, batch_size=64,
                        validation_split=0.3, callbacks=[early_stop], verbose=1)
    return model, history

def make_predictions(model, X, Y_true):
    """Make predictions using the trained model."""
    Y_pred = model.predict(X).reshape(-1)
    return Y_true, Y_pred

def calculate_metrics(y_true, y_pred):
    """Calculate MSE, RMSE, and MAE."""
    mse = mean_squared_error(y_true, y_pred)
    rmse = math.sqrt(mse)
    mae = mean_absolute_error(y_true, y_pred)
    return mse, rmse, mae

def plot_training_loss(loss_dict, stock_name, output_dir):
    """Plot the training loss vs epochs for each model variant."""
    plt.figure(figsize=(10, 6))
    plt.semilogy(loss_dict.get('Control', []), label='Control Model', linewidth=2, color='black')
    plt.semilogy(loss_dict.get('Vader', []), label='Vader Model', linewidth=2, color='red')
    plt.semilogy(loss_dict.get('BERT', []), label='Bert Model', linewidth=2, color='blue')
    plt.grid(True, which="both", linestyle='--', linewidth=0.5)
    plt.gca().yaxis.set_minor_formatter(plt.NullFormatter())
    plt.xlabel('Epoch')
    plt.ylabel('Loss (log scale)')
    plt.title('Training Loss of Different RNN Models')
    plt.legend()
    if not os.path.exists(output_dir):
        os.makedirs(output_dir)
    plt.savefig(os.path.join(output_dir, f'{stock_name}_model_training_loss.pdf'), format='pdf')
    plt.show()

def plot_test_results(test_dates, actual, predictions, stock_name, output_dir):
    """Create a 3-panel subplot of actual vs predicted, residuals over time, and residual KDE."""
    # Compute residuals for each model
    residuals = {}
    for key, pred in predictions.items():
        residuals[key] = actual - pred

    # Prepare DataFrame for KDE plotting of residuals
    residuals_df = pd.DataFrame(residuals)
    melted_res = residuals_df.melt(var_name='Model', value_name='Residuals')
    model_colours = {'Control': 'blue', 'Vader': 'green', 'BERT': 'red'}

    fig, axes = plt.subplots(1, 3, figsize=(18, 5))

    ax1 = axes[0]
    ax1.plot(test_dates, actual, label='Actual Price', color='black')
    ax1.plot(test_dates, predictions.get('Control', []), label='Control Model', color='blue')
    ax1.plot(test_dates, predictions.get('Vader', []), label='Vader Model', color='green')
    ax1.plot(test_dates, predictions.get('BERT', []), label='BERT Model', color='red')
    ax1.set_title(f'{stock_name} - Test Data - Actual vs Predicted')
    ax1.set_ylabel('Stock Price')
    ax1.grid(True)
    ax1.xaxis.set_major_locator(mdates.MonthLocator(interval=1))
    ax1.xaxis.set_major_formatter(mdates.DateFormatter('%b %y'))
    plt.setp(ax1.xaxis.get_majorticklabels(), rotation=45, ha="right")
    ax1.legend(loc='upper center', bbox_to_anchor=(0.5, -0.2), ncol=1, fontsize=11)

    ax2 = axes[1]
    ax2.axhline(0, color='black', linestyle='-', label='Ideal Residual')
    ax2.plot(test_dates, residuals.get('Control', []), label='Residual Control', color='blue')
    ax2.plot(test_dates, residuals.get('Vader', []), label='Residual Vader', color='green')
    ax2.plot(test_dates, residuals.get('BERT', []), label='Residual BERT', color='red')
    ax2.set_title(f'{stock_name} - Test Data - Residuals Over Time')
    ax2.set_ylabel('Residual (Actual - Predicted)')
    ax2.grid(True)
    ax2.xaxis.set_major_locator(mdates.MonthLocator(interval=1))
    ax2.xaxis.set_major_formatter(mdates.DateFormatter('%b %y'))
    plt.setp(ax2.xaxis.get_majorticklabels(), rotation=45, ha="right")
    ax2.legend(loc='upper center', bbox_to_anchor=(0.5, -0.2), ncol=1, fontsize=11)

    ax3 = axes[2]
    sns.kdeplot(
        data=melted_res,
```

```
        x='Residuals',
        hue='Model',
        fill=True,
        alpha=0.1,
        palette=model_colours,
        ax=ax3
    )
    for model, color in model_colours.items():
        mean_res = residuals_df[model].mean()
        ax3.axvline(mean_res, color=color, linestyle='--', label=f'{model} (Mean: {mean_res:.4f})')
    ax3.set_title(f'{stock_name} - Test Data - Residual Distribution')
    ax3.set_xlabel('Residuals')
    ax3.set_ylabel('Density')
    ax3.grid(True)
    ax3.legend(loc='upper center', bbox_to_anchor=(0.5, -0.2), ncol=1, fontsize=11)

    plt.tight_layout()
    if not os.path.exists(output_dir):
        os.makedirs(output_dir)
    plt.savefig(os.path.join(output_dir, f'residuals_{stock_name}.pdf'), format='pdf')
    plt.show()

def train_and_evaluate_asx200(max_epochs=250, patience=25, train_proportion=0.7, lookback=30, forecast=1,
                              control_columns=['Close'],
                              sentiment_columns={'Vader': ['vader_negative', 'vader_neutral', 'vader_positive', '
      vader_compound'],
                                                 'BERT': ['bert_sentiment_score', 'bert_sentiment_label']},
                              output_dir='results'):
    """Run the full analysis pipeline for the ASX200 index using preprocessed sentiment data."""
    # prepare sentiment data and plot article cont over time
    daily_sentiment = prepare_asx200_sentiment_data()
    plot_article_count(daily_sentiment)

    # determine the date range from sentiment data
    start_date = daily_sentiment.index.min().strftime('%Y-%m-%d')
    end_date = daily_sentiment.index.max().strftime('%Y-%m-%d')

    # get ASX200 index price data via yfinance
    price_data = get_asx200_price_data(start_date, end_date)

    # Merge price data with daily sentiment data
    merged_df = merge_asx200_data(price_data, daily_sentiment)

    # prepare dataset variants (Control, Vader, and BERT)
    data_variants = prepare_data(merged_df, control_columns, sentiment_columns)

    performance = []
    loss_dict = {}
    pred_dict = {}
    control_Y_true = None
    test_dates = None

    # For each variant, split the data, train the model, and record predictions/metrics
    for key, dataset in data_variants.items():
        X_train, Y_train, X_test, Y_test, scaler = train_test_split_data(dataset, train_proportion, lookback, forecast)
        input_shape = (X_train.shape[1], X_train.shape[2])
        model, history = build_and_train_model(X_train, Y_train, input_shape, max_epochs, patience)
        loss_dict[key] = history.history['loss']

        Y_true, Y_pred = make_predictions(model, X_test, Y_test)
        # For the control variant  record the test dates and actual values for plotting
        if key == 'Control':
            train_size = int(len(dataset) * train_proportion)
            test_data = dataset.iloc[train_size:]
            test_dates = test_data.index[lookback + forecast - 1:][:len(Y_test)]
            control_Y_true = Y_true

        mse, rmse, mae = calculate_metrics(Y_true, Y_pred)
        performance.append({'Model': key + '_ASX200', 'MSE': mse, 'RMSE': rmse, 'MAE': mae})
        pred_dict[key] = Y_pred

    # plot the training loss and test results using the custom formatting
    plot_training_loss(loss_dict, 'ASX200', output_dir)
    plot_test_results(test_dates, control_Y_true, pred_dict, 'ASX200', output_dir)

    return pd.DataFrame(performance)


performance_df = train_and_evaluate_asx200(max_epochs=50, train_proportion=0.7)  # Use fewer epochs for testing
performance_df
```